

Umm Al-Qura University

College of Computing

College of Computing Ontology

Final Project Report

DS2203 – Knowledge Representation & Inference

Student Name	Student ID	Role
Ebdaa Btawi	446004338	Leader
Razan Al-Rashdi	446001745	Member
Dana Al-Qurashi	446001073	Member
Shomokh Al-Malki	445000560	Member

Ontology File: final_project_fixed.owl

Tool: Protégé 5.x | Language: OWL 2 DL

Table of Contents

Task Distribution

Requirements Coverage Checklist

Section 1: Class Hierarchy

- 1.1 What is the TBox?
- 1.2 Design Methodology
- 1.3 Coverage of Required Categories
- 1.4 Class Hierarchy Screenshots

Section 2: Properties

- 2.1 Object Properties
- 2.2 Functional Properties
- 2.3 Data Properties

Section 3: Axioms and Constraints

- 3.1 Cardinality Restrictions
- 3.2 Disjoint Classes
- 3.3 Class Equivalence

Section 4: Individuals (ABox)

Section 5: Reasoning Results

Section 6: SPARQL Queries

Task Distribution

Student Name	Student ID	Responsibilities
Ebdaa Btawi	446004338	Task 1 (Class Hierarchy), Task 2 (Object Properties), Report Writing
Dana Al-Qurashi	446001073	Task 3 (Data Properties & Axioms), Report Writing
Razan Al-Rashdi	446001745	Task 4 (Individuals / ABox), Task 5 (Reasoning), Protégé screenshots, Report Writing
Shomokh Al-Malki	445000560	Task 6 (SPARQL Queries), Query execution & result screenshots, Report formatting

Requirements Coverage Checklist

Task	Requirement	Min	Delivered	Status
Task 1	OWL Classes (TBox)	≥ 10	25 classes	✓
Task 2	Object Properties	≥ 8	20 (with inverses)	✓
Task 2	Hierarchical relations	≥ 1	2	✓
Task 2	Participation relations	≥ 1	2	✓
Task 2	Organisational / spatial	≥ 1	2	✓
Task 2	Associative / support	≥ 1	4	✓
Task 2	Inverse property pairs	≥ 1	9 pairs	✓
Task 2	Functional properties	≥ 1	5	✓
Task 3	Data Properties	≥ 6	9	✓
Task 3	Cardinality Restriction	≥ 1	3 (exact 1 + min 1×2)	✓
Task 3	Disjoint Classes	required	7 declarations	✓
Task 3	Class Equivalence	3	3 defined	✓
Task 4	Individuals (ABox)	≥ 20	28 individuals	✓
Task 4	Object property assertions	required	All individuals	✓
Task 4	Data property assertions	required	All individuals	✓
Task 5	Reasoner consistency	required	HermiT — Consistent	✓
Task 5	Inferred memberships	required	7 inferences	✓
Task 6	SPARQL Queries	4 distinct	4 queries	✓
Task 6	Distinct SPARQL features	required	COUNT/GROUP BY, ASK, FILTER/REGEX, UNION	✓

Section 1: Class Hierarchy

1.1 What is the TBox?

In OWL 2, a knowledge base has two parts:

- TBox (Terminological Box) — defines classes and subclass relationships. Captures general knowledge such as 'every Department is a sub-unit of CollegeOfComputing'.
- ABox (Assertional Box) — defines individuals and their property values (Section 4).

The TBox models the College of Computing at UQU. All class names are based on real data from the college website (<https://uqu.edu.sa/en/ccomp>).

1.2 Design Methodology

The domain was analysed by reviewing the official college website and identifying the main entities: departments, committees, research groups, people, academic programs, courses, events, and facilities. These were organised into a meaningful hierarchy using `rdfs:subClassOf`.

1.3 Coverage of Required Categories

Required Category	OWL Classes Defined	Count
College as a whole	CollegeOfComputing	1
Sub-units and components	Department, Committee, ResearchGroup	3
People and roles	Person, FacultyMember, Dean, DepartmentHead, Student, UndergraduateStudent, GraduateStudent, AdministrativeStaff	8
Academic structures	AcademicProgram, BachelorProgram, MasterProgram, Course	4
Activities or events	Event, Hackathon, GraduationProject	3
Supporting structures	Facility, ComputerLab, Library	3

1.4 Class Hierarchy Screenshots



Figure 1 – Class hierarchy in Protégé 5.x (asserted view)



Figure 2 – Class hierarchy (inferred) after HermiT — yellow background confirms consistency

Section 2: Properties

2.1 Object Properties

Twenty object properties were defined covering all five required relation types.

#	ObjectProperty	InverseOf	Domain	Range	Type	Func ?
1	hasDepartment	isDepartmentOf	CollegeOfComputing	Department	Hierarchical	—
2	enrolledIn	hasStudent	Student	AcademicProgram	Participation	✓
3	locatedIn	hasFacility	Facility	CollegeOfComputing	Spatial	✓
4	teaches	isTaughtBy	FacultyMember	Course	Associative	—
5	offersProgram	isOfferedBy	Department	AcademicProgram	Organisational	✓
6	participatesIn	hasParticipant	Student	Event	Participation	—
7	belongsToResearchGroup	hasResearcher	FacultyMember	ResearchGroup	Support	—
8	headedBy	isHeadOf	Department	DepartmentHead	Hierarchical	✓
9	includesCourse	isPartOfProgram	AcademicProgram	Course	Support	—
10	worksFor	—	AdministrativeStaff	CollegeOfComputing	Organisational	—
11	hasMember	—	Committee	FacultyMember	Support	—



Figure 3 – Object property hierarchy in Protégé

2.2 Functional Properties — Justification

- enrolledIn — a Student is enrolled in exactly one AcademicProgram at a time.
- locatedIn — a Facility is physically located in exactly one CollegeOfComputing.
- isOfferedBy — each AcademicProgram is owned by exactly one Department.
- headedBy — each Department has exactly one DepartmentHead.
- isDepartmentOf — each Department belongs to exactly one college.

Note: isTaughtBy was excluded from Functional because a course may be co-taught by more than one instructor.

2.3 Data Properties

Data Property	Domain	Range	Description
courseTitle	Course / AcademicProgram	xsd:string	Official name of the course or program
creditHours	Course	xsd:integer	Number of credit hours
email	Person	xsd:string	Official university email address
establishedYear	Department / ResearchGroup	xsd:integer	Year the unit was founded
hasID	Person	xsd:integer	Unique numerical ID
officeNumber	FacultyMember / Facility	xsd:string	Office or room identifier (e.g. B201)
phoneNumber	Person	xsd:string	Contact number (string to preserve + prefix)
websiteURL	CollegeOfComputing	xsd:anyURI	Web address of the college
eventDate	Event	xsd:date	Date of the event

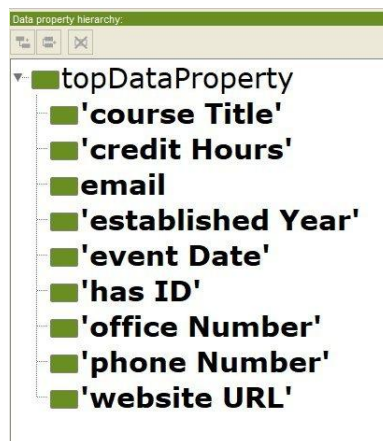


Figure 4 – Data property hierarchy in Protégé

Section 3: Axioms and Constraints

3.1 Cardinality Restrictions

Exact Cardinality — Department

```
Department  $\sqsubseteq$  (headedBy exactly 1 DepartmentHead)
```

Every department must be headed by exactly one DepartmentHead. The reasoner validates that no department lacks a head or has multiple heads.

Minimum Cardinality — Academic Programs

```
BachelorProgram  $\sqsubseteq$  (includesCourse min 1 Course)
```

```
MasterProgram  $\sqsubseteq$  (includesCourse min 1 Course)
```

Every degree program must include at least one course, preventing empty programs.

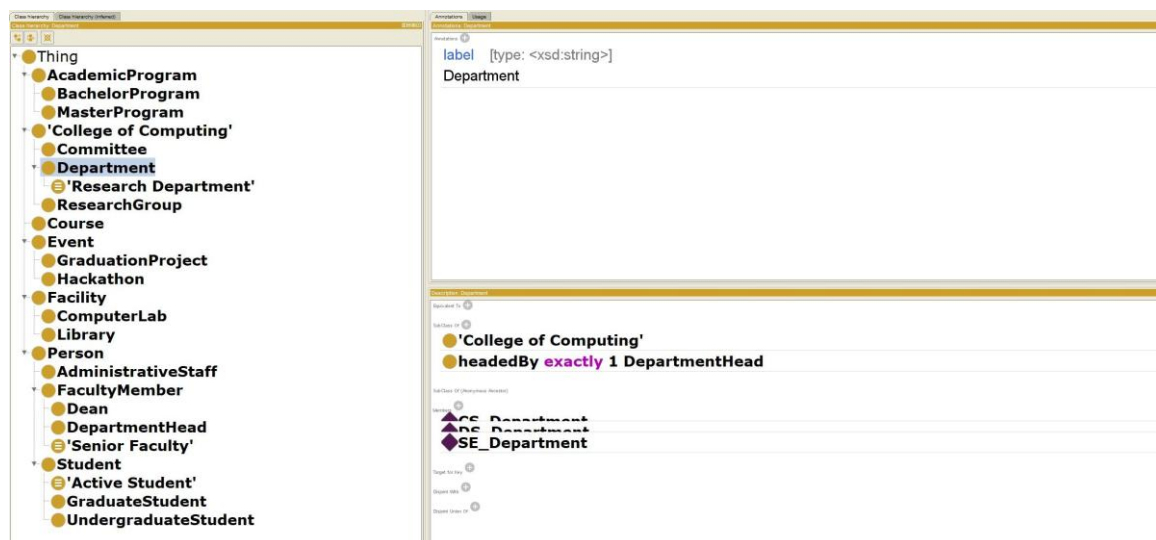


Figure 5 – Department showing headedBy exactly 1 DepartmentHead cardinality

3.2 Disjoint Classes

Disjointness axioms declare that two classes share no common individuals.

Level	Disjoint Declaration	Rationale
1	$\text{Course} \sqcap \text{Person} = \perp \mid \text{Event} \sqcap \text{Person} = \perp$	A person cannot be a course or event.
2	$\text{FacultyMember} \sqcap \text{Student} = \perp$	A faculty member cannot simultaneously be a student.
3	$\text{BachelorProgram} \sqcap \text{MasterProgram} = \perp$	A program cannot be both undergraduate and postgraduate.
4	$\text{GraduateStudent} \sqcap \text{UndergraduateStudent} = \perp$	A student is enrolled at one level only.
5	$\text{ComputerLab} \sqcap \text{Library} = \perp$	A lab cannot be a library.

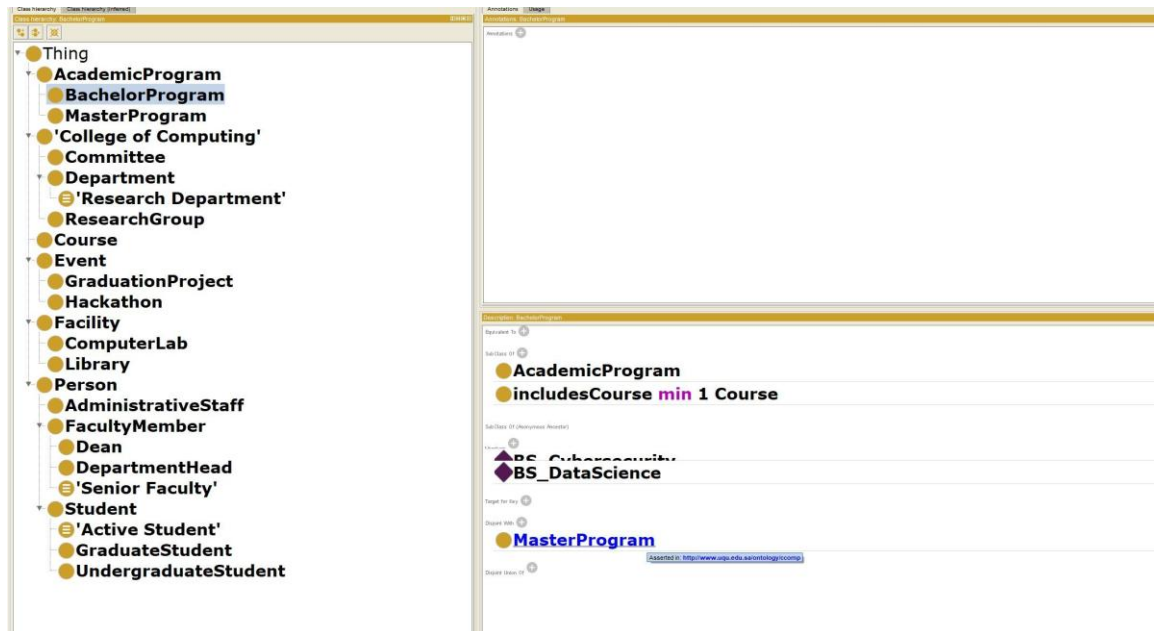


Figure 6 – BachelorProgram showing DisjointWith MasterProgram

3.3 Class Equivalence (Necessary & Sufficient Conditions)

Three classes were defined using EquivalentTo axioms. Any individual satisfying all conditions is automatically classified by the reasoner.

Equivalence 1 — SeniorFaculty

```
SeniorFaculty ≡ FacultyMember
    and (belongsToResearchGroup some ResearchGroup)
    and (teaches some Course)
```

Inferred individuals: Dr_Ahmad, Dr_Khalid, Dr_Nora

Equivalence 2 — ActiveStudent

```
ActiveStudent ≡ Student
    and (enrolledIn some AcademicProgram)
    and (participatesIn some Event)
```

Inferred individuals: Student_Ghala, Student_Lama, Student_Razan

Equivalence 3 — ResearchDepartment

```
ResearchDepartment ≡ Department
    and (offersProgram some MasterProgram)
```

Inferred individual: CS_Department

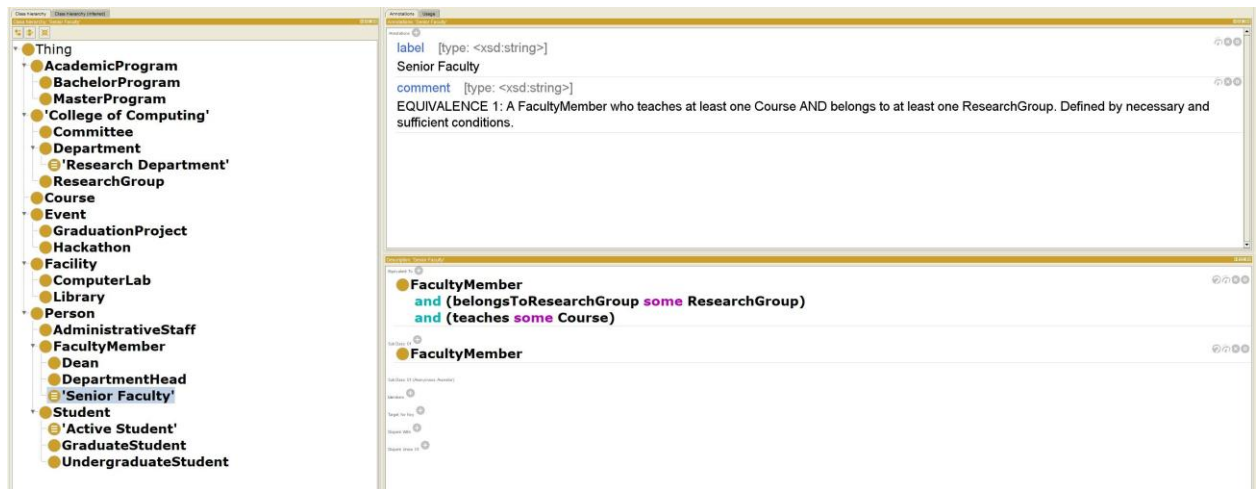


Figure 7 – SeniorFaculty equivalence axiom in Protégé

Section 4: Individuals (ABox)

The ontology contains 28 individuals covering all main classes.

Class	Individuals	Count
BachelorProgram	BS_DataScience, BS_Cybersecurity	2
MasterProgram	MS_ComputerScience	1
CollegeOfComputing	CComp_College	1
Department	CS_Department, DS_Department, SE_Department	3
Course	AI_Course, CyberSecurity_Course, KRI_Course, Programming_Course	4
Committee	Academic_Committee, Research_Committee	2
GraduationProject	GradProject2026	1
Hackathon	Hackathon2026	1
ComputerLab	AI_Lab	1
Library	Main_Library	1
ResearchGroup	AI_Research_Group, Cyber_Research_Group	2
AdministrativeStaff	Huda_Staff	1
FacultyMember / DepartmentHead	Dr_Ahmad, Dr_Khalid, Dr_Nora, Head_SE_Department	4
Dean	Dr_Sara	1
GraduateStudent	Student_Razan	1
UndergraduateStudent	Student_Ghala, Student_Lama	2
TOTAL		28

4.1 Key Individual Examples

- Dr_Ahmad — DepartmentHead of CS_Department, teaches KRI_Course, belongs to AI_Research_Group. Email: ahmad@uqu.edu.sa, Office: B201.
- Dr_Khalid — FacultyMember, teaches AI_Course, belongs to AI_Research_Group.
- Dr_Nora — DepartmentHead of DS_Department, teaches CyberSecurity_Course, Cyber_Research_Group.
- Student_Ghala — UndergraduateStudent, enrolledIn BS_DataScience, participatesIn Hackathon2026.
- Student_Razan — GraduateStudent, enrolledIn MS_ComputerScience, participatesIn Hackathon2026.
- Huda_Staff — AdministrativeStaff, worksFor CComp_College.



Figure 8 – All 28 individuals in the Protégé Individuals panel

Section 5: Reasoning Results

The HermiT reasoner (v1.4.3.456) was used in Protégé 5.x to verify ontology consistency and infer new knowledge.

5.1 Consistency Verification

- No individual is simultaneously asserted to two disjoint classes.
- Department headedBy exactly 1 DepartmentHead — satisfied for all three departments.
- includesCourse min 1 Course — satisfied for all BachelorProgram and MasterProgram individuals.
- No owl:Nothing inferred under any class — all classes are satisfiable.



Figure 9 – Inferred class hierarchy confirming consistency (yellow = reasoner active)

5.2 Inferred Class Memberships

Defined Class	Inferred Individual	Reason
SeniorFaculty	Dr_Ahmad	FacultyMember + belongsToResearchGroup(AI_Research_Group) + teaches(KRI_Course)
SeniorFaculty	Dr_Khalid	FacultyMember + belongsToResearchGroup(AI_Research_Group) + teaches(AI_Course)
SeniorFaculty	Dr_Nora	FacultyMember (via DepartmentHead) + belongsToResearchGroup + teaches
ActiveStudent	Student_Ghala	Student + enrolledIn(BS_DataScience) + participatesIn(Hackathon2026)

ActiveStudent	Student_Lama	Student + enrolledIn(BS_Cybersecurity) + participatesIn(GradProject2026)
ActiveStudent	Student_Razan	Student + enrolledIn(MS_ComputerScience) + participatesIn(Hackathon2026)
ResearchDepartment	CS_Department	Department + offersProgram(MS_ComputerScience : MasterProgram)

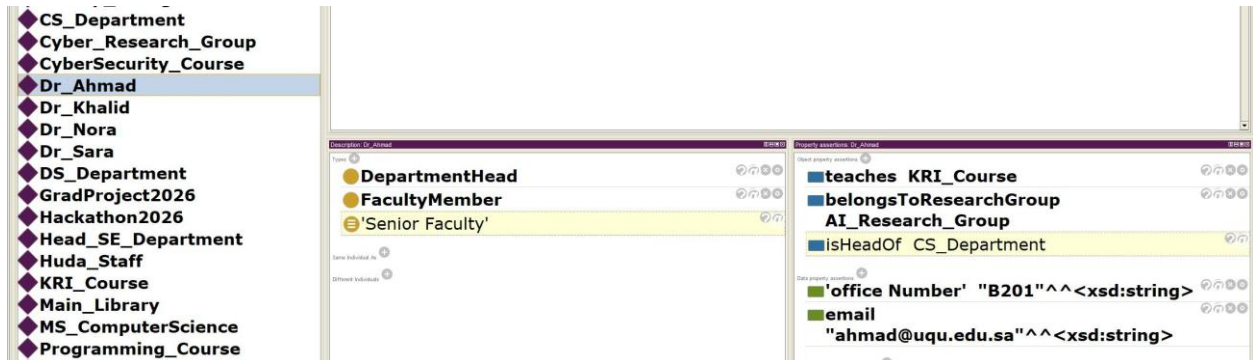


Figure 10 – Dr_Ahmad showing inferred type 'Senior Faculty' alongside asserted types

5.3 Inferred Property Relationships

- teaches(Dr_Ahmad, KRI_Course) → inferred isTaughtBy(KRI_Course, Dr_Ahmad)
- enrolledIn(Student_Ghala, BS_DataScience) → inferred hasStudent(BS_DataScience, Student_Ghala)
- headedBy(CS_Department, Dr_Ahmad) → inferred isHeadOf(Dr_Ahmad, CS_Department)

Section 6: SPARQL Queries

Query 1 — SELECT with Aggregation (COUNT + GROUP BY)

Purpose

Retrieve each department with the number of programs it offers and its founding year. Demonstrates aggregate functions and OPTIONAL.

SPARQL Features Used

SELECT with COUNT | GROUP BY | ORDER BY DESC | OPTIONAL

Query

```
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
SELECT ?dept (COUNT(?prog) AS ?numPrograms) ?foundedYear
WHERE {
  ?dept a :Department .
  ?dept :offersProgram ?prog .
  OPTIONAL { ?dept :establishedYear ?foundedYear }
}
GROUP BY ?dept ?foundedYear
ORDER BY DESC(?numPrograms)
```

Result

SPARQL query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
SELECT ?dept (COUNT(?program) AS ?numPrograms) ?foundedYear
WHERE {
  ?dept a :Department .
  ?dept :offersProgram ?program .
  OPTIONAL { ?dept :establishedYear ?foundedYear }
}
GROUP BY ?dept ?foundedYear
ORDER BY DESC(?numPrograms)

```

dept	numPrograms	foundedYear
CS_Department	"2" ^{AA} <http://www.w3.org/2001/XMLSchema#in: "2010" ^{AA} <xsd:integer>	
DS_Department	"1" ^{AA} <http://www.w3.org/2001/XMLSchema#in: "2020" ^{AA} <xsd:integer>	

Execute

Figure 11 – Query 1: CS_Department (2 programs, 2010), DS_Department (1 program, 2020)

Query 2 — ASK Query (Boolean Verification)

Purpose

Verify whether a graduate student enrolled in a Master's program also participates in an event.

SPARQL Features Used

ASK query form | Boolean result | Multiple triple patterns

Query

```

PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
ASK {
  ?student a :GraduateStudent .
  ?student :enrolledIn ?prog .
  ?prog a :MasterProgram .
  ?student :participatesIn ?event .
  ?event a :Event .
}

```

Result & Discussion

Result: False under plain SPARQL 1.1. Student_Razan satisfies the first four conditions but Hackathon2026 is typed as :Hackathon (subclass of :Event), not directly as :Event. Plain SPARQL does not follow subclass chains. Under OWL 2 RL Entailment Regime with the reasoner active, the result would be True. This demonstrates entailment regime behaviour, not a modelling error.

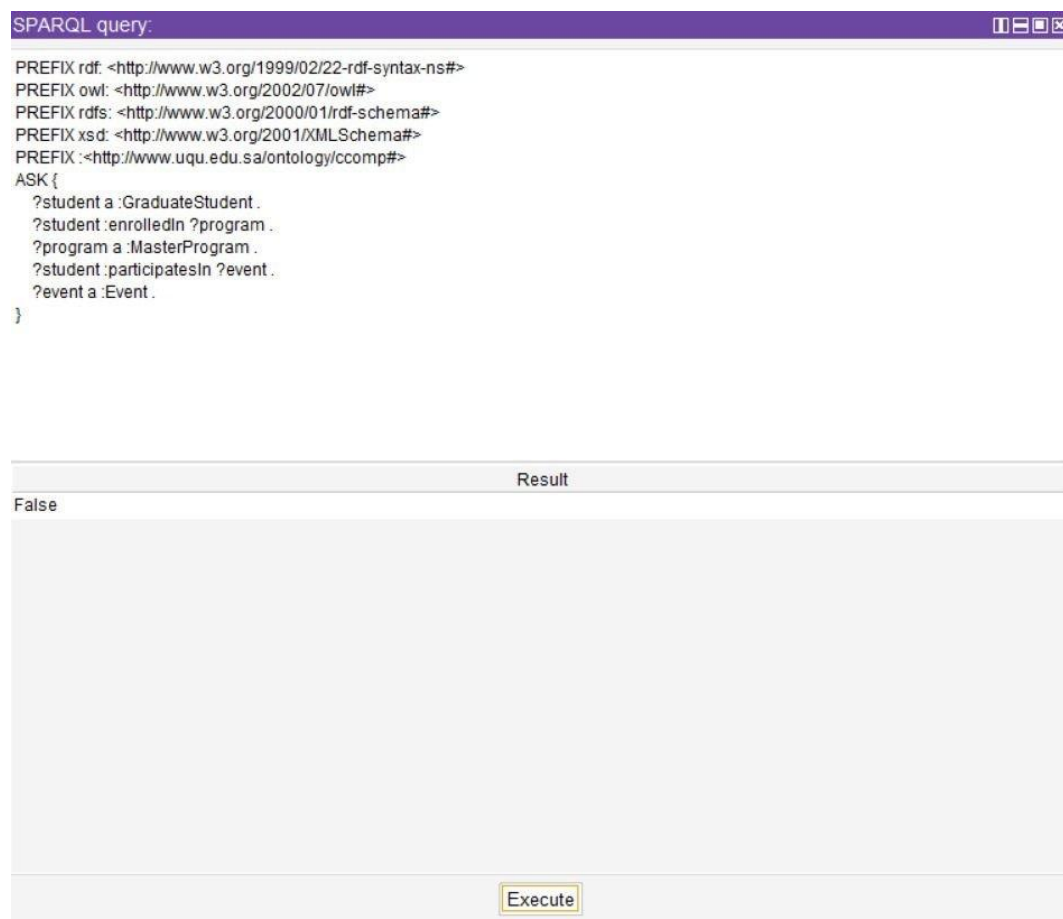


Figure 12 – Query 2 result: False (expected under plain SPARQL entailment)

Query 3 — SELECT with FILTER and REGEX

Purpose

Retrieve all courses whose titles begin with 'A' or 'C', along with their credit hours.

SPARQL Features Used

SELECT | FILTER with regex() | ORDER BY

Query

```
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
```

```

SELECT ?course ?title ?hours
WHERE {
  ?course a :Course .
  ?course :courseTitle ?title .
  ?course :creditHours ?hours .
  FILTER(regex(?title, '^[AC]', 'i'))
}
ORDER BY ?title

```

Result

SPARQL query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
SELECT ?course ?title ?hours
WHERE {
  ?course a :Course .
  ?course :courseTitle ?title .
  ?course :creditHours ?hours .
  FILTER (regex(?title, "^(A|C)"))
}
ORDER BY ?title

```

course	title	hours
AI_Course	"Artificial Intelligence"	"3"^^<xsd:integer>
CyberSecurity_Course	"Cybersecurity Fundamentals"	"3"^^<xsd:integer>

Execute Search

Figure 13 – Query 3: AI_Course and CyberSecurity_Course (both 3 credit hours)

Query 4 — SELECT with UNION

Purpose

Retrieve all faculty including FacultyMember and DepartmentHead individuals with courses they teach and research groups.

SPARQL Features Used

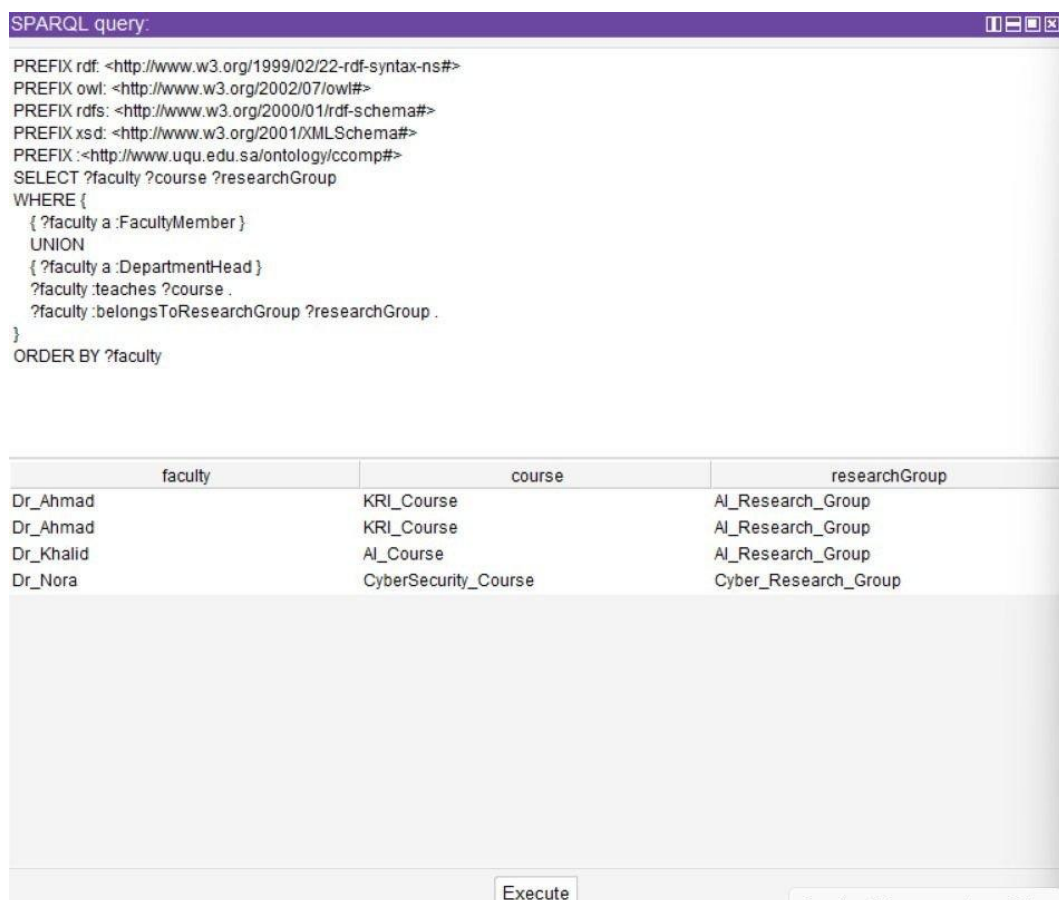
SELECT | UNION (two class patterns) | ORDER BY | Multiset semantics

Query

```
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
SELECT ?faculty ?course ?researchGroup
WHERE {
  { ?faculty a :FacultyMember }
  UNION
  { ?faculty a :DepartmentHead }
  ?faculty :teaches ?course .
  ?faculty :belongsToResearchGroup ?researchGroup .
}
ORDER BY ?faculty
```

Result & Discussion

The query returns 4 rows. Dr_Ahmad appears twice because UNION is not deduplicated by default — he matches both FacultyMember and DepartmentHead patterns. This demonstrates SPARQL multiset semantics. Adding SELECT DISTINCT would collapse Dr_Ahmad to one row. Dr_Nora appears once showing the full faculty coverage including department heads.



The screenshot shows a SPARQL query editor window with a purple title bar. The query is as follows:

```
SPARQL query:
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX : <http://www.uqu.edu.sa/ontology/ccomp#>
SELECT ?faculty ?course ?researchGroup
WHERE {
  { ?faculty a :FacultyMember }
  UNION
  { ?faculty a :DepartmentHead }
  ?faculty :teaches ?course .
  ?faculty :belongsToResearchGroup ?researchGroup .
}
ORDER BY ?faculty
```

Below the query, the results are displayed in a table with three columns: faculty, course, and researchGroup. The results are as follows:

faculty	course	researchGroup
Dr_Ahmad	KRI_Course	AI_Research_Group
Dr_Ahmad	KRI_Course	AI_Research_Group
Dr_Khalid	AI_Course	AI_Research_Group
Dr_Nora	CyberSecurity_Course	Cyber_Research_Group

At the bottom of the window, there is an 'Execute' button and a 'Results' button.

Figure 14 – Query 4: Dr_Ahmad appears twice (UNION multiset), Dr_Khalid and Dr_Nora once each

6.5 SPARQL Summary

#	Type	Key Features	Purpose
1	SELECT	COUNT, GROUP BY, ORDER BY DESC, OPTIONAL	Programs per department with founding year
2	ASK	Boolean result, entailment regime discussion	Graduate student event participation
3	SELECT	FILTER, regex(), ORDER BY	Courses starting with A or C
4	SELECT	UNION, multiset semantics, ORDER BY	Faculty teaching and research groups